# Navigal: Generating Adaptive Interfaces for Personalized Screen Reader Navigation

ANONYMOUS AUTHOR(S)

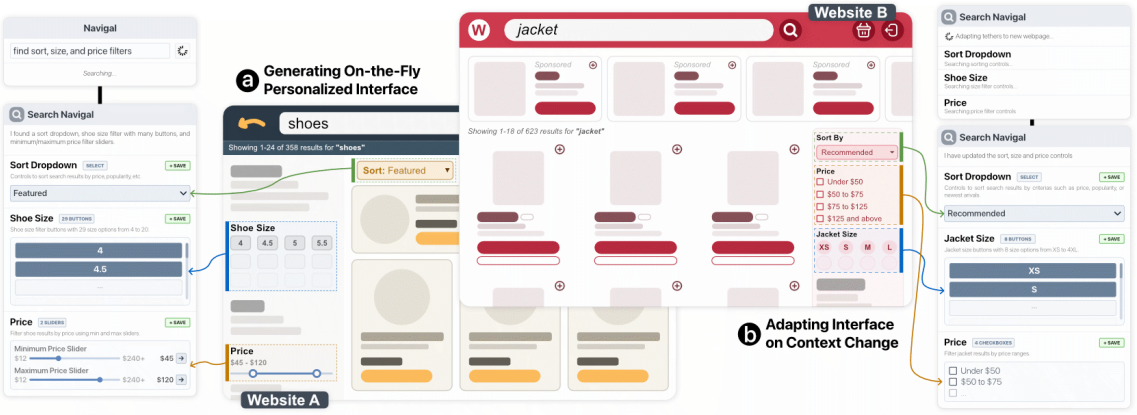



Fig. 1. Every webpage has a unique layout, requiring screen reader users to sequentially traverse each page's structure to locate and access the controls they need, and to relearn where things are when moving between sites. Navigal enables users to (a) **generate an on-the-fly personalized interface** by stating in natural language which controls they are looking for; the system searches the page for matching controls and constructs a panel of *tethers* that are directly linked to the corresponding elements on the website. Furthermore, (b) as the user navigates to a different page, Navigal re-searches for the requested controls and **adapts each tether to the newly available elements**, providing a consistent navigation experience across sites.

Screen reader users navigate the web by linearly traversing elements, often relying on memorized page layouts that can become unreliable on unfamiliar websites, making navigation both cognitively demanding and brittle. While non-linear navigation methods such as rotor controls and window spots exist, they are either generic or system-defined, offering limited support for quick, repeated access to goal-relevant controls. We introduce *Navigal*, a browser extension that generates personalized, goal-specific interfaces on-demand for efficient screen reader navigation. These interfaces bind to existing web page controls, persist across pages, and dynamically adapt to new web contexts, allowing users to simply describe what they need and reuse the same interface as they navigate. A technical evaluation showed Navigal supported 18 of 20 web tasks and reduced keystrokes by 80% over linear traversal. In an exploratory study with 7 blind users, Navigal significantly reduced navigational effort and cognitive load compared to unassisted screen reader use.

CCS Concepts: • **Human-centered computing** → **Accessibility systems and tools**.

Additional Key Words and Phrases: Screen Readers; Web Accessibility; Adaptive Interfaces; Blind and Low Vision; Large Language Models

**ACM Reference Format:**

Anonymous Author(s). 2026. Navigal: Generating Adaptive Interfaces for Personalized Screen Reader Navigation. 1, 1 (September 2026), 27 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Non-visual navigation of graphical user interfaces (GUIs) remains a central challenge in accessible computing. GUIs are built around visual affordances like spatial layout, grouping, and visual hierarchy that allow sighted users to rapidly perceive structure and locate relevant controls. Screen reader users primarily access the same interfaces sequentially, element by element, and must build a mental model of page structure before they can navigate efficiently [15]. Users often develop site-specific navigation strategies for websites they use regularly to mitigate this cost [36]. Even so, non-visual navigation is fundamentally constrained by the speed of sequential information processing [53], as users must attentively listen to and evaluate each intermediate element to reach their target, making the process cognitively demanding even on familiar pages. This is further exacerbated on unfamiliar websites, where users may not know whether the target they seek is present, difficult to locate, or absent entirely until they have traversed the page [11].

Prior work has explored various approaches to this problem. Screen readers offer structural shortcuts that let users jump between headings, landmarks, and form fields, but these depend on well-authored markup and produce filtered sequential traversal rather than goal-directed access [15, 53]. Research systems have reduced what users must traverse through content segmentation, summarization, and semantic abstraction [3, 7, 24]. End-user customization frameworks have let users script and share accessibility fixes for specific pages [9, 27, 66]. In each case, the user still interacts within the page's existing element structure, and the effort invested in learning one page does not accumulate or transfer to the next.

Recent Large Language Model (LLM)-based approaches have applied language models to improve non-visual web access, whether by filtering page content to surface task-relevant elements [23], restructuring page hierarchies for clearer screen reader traversal [64], or executing interface actions on behalf of users through natural language commands [35]. Filtering and restructuring reduce what users must attend to, but interaction still takes place within the page's existing element structure. A user searching for a specific product attribute must still locate and operate whatever control the website provides, however that control was designed. Fully agentic approaches [35, 42] remove this burden by automating interaction entirely, but also reduce user's direct control and make verification difficult for screen reader users who cannot visually monitor the agent's actions [2, 20]. Existing systems therefore either help users navigate the interface that exists, or replace navigation by acting on users' behalf. There is thus a growing need for approaches that enable screen reader users to define how they navigate and interact with visual interfaces, while retaining control over execution [27, 44].

In this paper, we introduce *Navigal*[1], an intelligent screen reader extension that enables screen reader users to define personalized interaction layers over the GUIs they interact with. Rather than requiring users to learn and navigate each page's structure, Navigal lets them describe the controls they need in natural language and assembles them into a single, persistent interface that can adapt as the underlying context changes. Consider a screen reader user shopping for shoes online who needs to repeatedly move between price, size, and color filters and the search results, controls that are typically scattered across different regions of the page. With Navigal, the user queries for these controls, and the system generates an ad-hoc interface containing *tethers*, semantic bindings to the relevant page elements that serve as interaction proxies [66]. Each tether presents a simplified accessible version of its source element (e.g., the size filter appears as a group of checkboxes), and selecting an option transmits the change to the actual page. Because each tether is grounded to a specific location in the DOM, users can jump to the source element at any time to verify the generated control corresponds to the intended page element. Tethers can also adapt as context changes: if the user switches from

[1] **Navigal** is short for **N**avigation **Al**ly

shoes to clothing, the size filter automatically reflects categorical options like S, M, and L instead of numeric sizes. This adaptation can extend across sites. When the user navigates to a different shopping website, Navigal rebinds the saved tethers to the corresponding controls. Despite the variability of web interfaces in structure and layout, Navigal provides a stable interaction layer that lets users operate through familiar, self-defined controls even as underlying interfaces change.

Using Navigal as a platform, we present an interaction paradigm in which screen reader users construct personlized interaction layers around their goals rather than navigate each page's structure as given. We first evaluated Navigal on 20 everyday web tasks across 19 websites, where Navigal supported 18 tasks and reduced the navigation steps required to reach target elements from a median of 39 to 6. We evaluated Navigal in an exploratory study with 7 blind participants, comparing it against unassisted screen reader use on online shopping tasks. Participants using Navigal reported lower cognitive load and navigational effort, and all 7 expressed a desire to continue using the system. Participants described use cases from their own daily browsing where they would want to create personalized interfaces, and highlighted the ability to verify AI-generated controls against their source elements as important to their trust in the system.

In summary, this work makes the following contributions:

- An interaction paradigm for non-visual web access in which users construct intent-defined interaction layers that decouple interaction from page structure while preserving user control.
- Navigal, a system that instantiates this paradigm through semantically grounded tethers that can persist, adapt across web contexts, and remain directly verifiable against their source elements.
- Findings from a technical evaluation across 20 everyday web tasks and an exploratory study with blind users, showing that generated interaction layers can reduce navigational effort while supporting user control and source-level verification.

## 2 Related Work

This work draws on prior systems and studies that support efficient and goal-directed web access for blind users. We organize this work into three primary bodies of literature: non-visual navigation techniques, customization in assistive technology, and AI-mediated screen reader navigation.

### 2.1 Non-Visual Web Navigation

Screen reader users access web content by traversing the DOM in linear order, moving through elements in the sequence the author arranged them [15, 36], making the process both time-consuming and cognitively demanding. Built-in structural shortcuts such as heading jumps, landmark navigation, and element lists reduce this burden by letting users skip between elements of a chosen type [15, 59], and survey evidence identifies heading navigation as the most common strategy among experienced users [59]. While these shortcuts rely on the consistent use of semantic HTML and ARIA, large-scale audits reveal that the majority of popular websites still contain substantial accessibility defects [58, 60]. Furthermore, studies with blind users found that only about half of the problems they encounter on real websites are addressed by existing accessibility guidelines [45]. Users compensate for these gaps through site-specific strategies [53, 57], yet they often remain unable to tell whether content is inaccessible, difficult to locate, or absent from the page entirely [11].

To lower this navigational cost, prior research has proposed various techniques for reshaping page content for screen readers. Transcoding and restructuring systems reorder the DOM to foreground salient regions [5, 24, 63].

Summarization and skimming systems compress page sections into shorter auditory units or surface relevant content for non-linear access [3, 25, 38]. A separate body of work pairs physical input devices with screen readers and screen magnifiers to give users finer control over navigation [12, 20]. These approaches reduce traversal effort, and each accommodation remains bounded by what the original page structure exposes.

Another line of work raises screen reader interaction from individual elements to semantic units and tasks. TrailBlazer lets blind users record, share, and replay scripts of web actions, and helps users build scripts for new tasks by suggesting likely next steps [10]. SRAA groups related HTML elements into web entities such as search results, menus, and forms, and automates screen reading actions over these entities on demand [7]. SuggestOmatic represents a user's browsing history as operations over these entities, predicts the next action, and extends its suggestions to similar websites [6]. CrowdIntent offers keyboard commands for features common across websites, such as logging in or finding a store, and resolves each command to the matching link on the current site through machine learning classifiers and crowdsourced input [8]. These systems let knowledge from one page inform interaction on the next, in the form of recorded procedures, action histories, or a shared command vocabulary. In each case, users act through a vocabulary of steps, entities, or commands defined by the system, and each one resolves to a control the page author created, in the form the author built it. Letting users define the controls themselves and carry those controls from one site to the next remains unaddressed.

### 2.2 Customization in Assistive Technology

Prior work has explored mechanisms that allow screen reader users to author their own accessibility fixes through end-user customization. Commercial screen readers support this by exposing scripting languages and plugin frameworks for custom hotkeys, content reordering, and task automation [40]. Web automation tools such as Chickenfoot let users customize rendered pages through scripts that refer to elements by their visible labels [14]. Community-driven efforts have further extended these mechanisms into collaborative authorship, where users and volunteers share scripts and annotations that repair accessibility defects on specific websites [9, 33, 52]. Qualitative studies show that blind users regularly combine multiple tools to construct personalized workflows that no single tool supports [27], and recent end-user programming environments have begun to offer first-class tooling for such authorship in adjacent domains [28]. However, these approaches place users in the role of designers, and the cost of entry is high. Effective authorship typically requires programming skills, and the resulting customizations are tightly coupled to specific pages and break when those websites undergo even minor changes [40]. This fragility is particularly problematic because blind users describe their browsing as goal-directed and context-dependent, with information needs that shift across sites and sessions [1, 32, 49].

A separate HCI tradition has pursued adaptive interfaces that automatically adjust to the user without requiring manual authorship. Optimization-based generators select widgets and layouts against formal models of user ability [21, 22]. The ability-based design framework argues that systems should treat user capability as the design target rather than requiring users to conform to the interface [61]. Malleable interface systems let users reshape application surfaces directly, including through generative models that synthesize interactive components on demand [17, 34, 50, 56]. Optimization-based generators decide adaptations on the user's behalf, and malleable systems are largely built around visual, direct manipulation of the interface surface, leaving limited room for screen reader users to direct how an interface adapts to them.

Other systems insert an accessible layer between an existing interface and its user. Within accessibility research, interaction proxies introduced the architectural pattern of inserting a re-mapping layer between an application's interface and the user, hosting accessible substitutes for inaccessible native controls [66]. InSupport extracts data

Table 1. Design space of Navigal.

| Approach | Intent input | Interaction surface | Interface author | Persistent customization | Cross-site adaptation | Verification |
|---|---|---|---|---|---|---|
| Page modification [23, 24, 64] | ○ | Adapted page | System | ○ | ○ | ○ |
| Semantic navigation [6–8, 10] | ○ | Commands / page | System | ○ | ✓ | ○ |
| Scripted customization [9, 14, 18] | — | Modified page | Programmer | ✓ | ○ | n/a |
| Interface remapping [19, 55, 66] | — | Proxy controls | Programmer / system | ○ | ○ | n/a |
| UI agents [30, 35, 42] | ✓ | Agent interface | Agent | n/a | ○ | ○ |
| **Navigal (this work)** | ✓ | **Proxy controls** | **User + AI** | ✓ | ✓ | ✓ |

✓ reported support; ○ varies across cited systems; n/a not reported; — outside the described interaction.

records such as search results from web pages and presents them through a proxy interface for efficient screen reader navigation [19]. AccessBolt identifies the controls of a desktop application and presents them on demand in a uniform dashboard that users navigate the same way across applications [55]. Commercially, accessibility overlays are third-party tools that website owners install to automatically repair accessibility barriers for all visitors to a site. However, blind and low vision users report that these tools often conflict with their screen readers and worsen existing barriers [39]. Across these layers, a developer, the system, or the website owner decides which accessible substitutes exist and how they behave, and every user receives the same set regardless of their individual goals.

Scripting gives users full control but demands programming skills they may not have, while system-generated adaptation removes that burden but sacrifices user control. Our work seeks a middle ground by leveraging AI to bridge this gap, enabling users to author interaction proxies without the need for manual programming. By treating the semantic intent of a control as the persistent anchor, Navigal allows these user-defined adaptations to remain resilient across the websites they visit.

### 2.3 AI-Mediated Screen Reader Navigation

Recent advances in LLMs have introduced new opportunities for AI-mediated navigation, where an AI system is positioned between the user and the webpage. Filtering approaches score page elements by relevance to a user-specified goal and suppress irrelevant content [23]. Regeneration approaches rewrite page markup into simplified structures with cleaner heading hierarchies [64]. Agentic approaches interpret natural-language instructions and execute interface actions on the user's behalf, utilizing either accessibility trees [35, 55] or vision-language models that operate on rendered pages [37, 67]. Finally, conversational approaches describe content or guide users through next steps while leaving execution to the user [43, 44]. While each approach reduces traversal effort, they do so at different costs to what users perceive and how much control they retain.

The costs of such mediation are beginning to surface in studies of how blind users engage with AI-powered tools. Users often approach generated outputs with calibrated skepticism, prioritizing designs that preserve their autonomy over the interaction [2, 4, 54]. They frequently prefer accessing original content over relying on opaque summaries

or automated actions that they cannot verify [43]. Consequently, mixed-initiative arrangements in which users can pause, inspect, or override the system outperform fully autonomous ones in both task success and user trust [29, 30, 42]. Filtering systems carry a silent failure mode in which content the model judged irrelevant is hidden without the user's awareness. Agentic systems remove the user from critical decision points in their own tasks [42]. Conversational systems preserve agency in execution but still require users to locate and operate each described control within the page. In each case, the user accepts reduced visibility into or reduced control over their own browsing in exchange for reduced effort.

A fundamental tradeoff therefore persists in recent AI-mediated accessibility tools. Approaches that reduce interaction effort do so by distancing the user from parts of the interaction, while approaches that preserve user control still route interaction through the same page structures that made traversal costly in the first place. Screen reader users are forced to choose which aspect of their agency to relinquish, as they lack tools to define how they would prefer to access a page on their own terms.

## 3 Navigal: Navigation Ally

Navigal introduces a new interaction model in which users operate through a personalized, generated interaction layer rather than navigating page structure directly. Instead of discovering and manipulating controls as arranged by the interface, users express their intent and interact through system-generated controls that are semantically bound to underlying page elements. This decouples interaction from page structure while preserving verifiability and user control.

### 3.1 Formative Sessions and Design Considerations

We conducted three remote formative sessions with two blind screen reader users with expertise in assistive technology, an assistive technology instructor and an assistive technology researcher. The sessions were unstructured interviews. We first discussed common web access problems and the experts' experiences with and concerns about AI-based computer access tools. We then introduced the premise of Navigal and gathered their input on its interaction design, including summoning the interface through a hotkey, focus behavior, and the overall interaction flow. Insights from these sessions directly shaped the design of Navigal's interface.

Table 1 summarizes how prior work and Navigal compare. From the gaps identified in prior work and the formative sessions, we derive four design considerations that guided Navigal's development.

**DC1: Intent-driven access.** Screen reader users treat exhaustive sequential traversal as a costly last resort, developing elaborate workarounds to approximate goal-directed browsing [15]. The system should let users specify what they want to reach and surface it directly, rather than requiring them to step through elements.

**DC2: Personalized control generation.** Web controls are frequently complex, poorly labeled, or inoperable via screen reader commands [36]. The system should produce simplified, accessible controls derived from the underlying DOM, so users interact with purpose-built components rather than native markup.

**DC3: Adaptive semantic bindings.** Screen reader scripts and bookmarks anchor to brittle text offsets that break on dynamic pages and fail to transfer across tools [40]. The system should maintain semantic bindings that persist across sessions, survive page mutations, and adapt when users move between sites.

**DC4: Verifiable AI mediation.** AI-powered UI agents increasingly automate web tasks end-to-end, but typically execute without involving users in critical choices, reducing agency [42]. Users also develop site-specific strategies for pages they visit regularly [53, 57], and tools that alter the page, such as accessibility overlays, often conflict with their

screen readers [39]. The system should preserve the original page as users already know it, act on the page only when users activate a control, and ground every generated control to its source element so users can verify it themselves.

### 3.2 Navigal Interactions

Navigal introduces the *tether*, an automatically generated accessible control that acts as an interaction proxy [66] for targeted element(s) on the underlying web page. Rather than requiring users to locate and interact with page elements directly, a tether provides a simplified, screen-reader-friendly control that propagates user actions to the bound page element on their behalf. Each tether carries a label, a goal-aware description that reflects the user's intent (e.g., "Language selector, currently set to English, 12 options available"), and a navigation link to its source element for direct verification against the actual page state. We support a range of tether types covering common web interaction patterns, including buttons, checkboxes, text inputs, sliders, select menus, radio groups, static content elements, and container summaries (the full set is described in Appendix E). We now describe how users create, interact with, and benefit from tethers through five interaction examples.

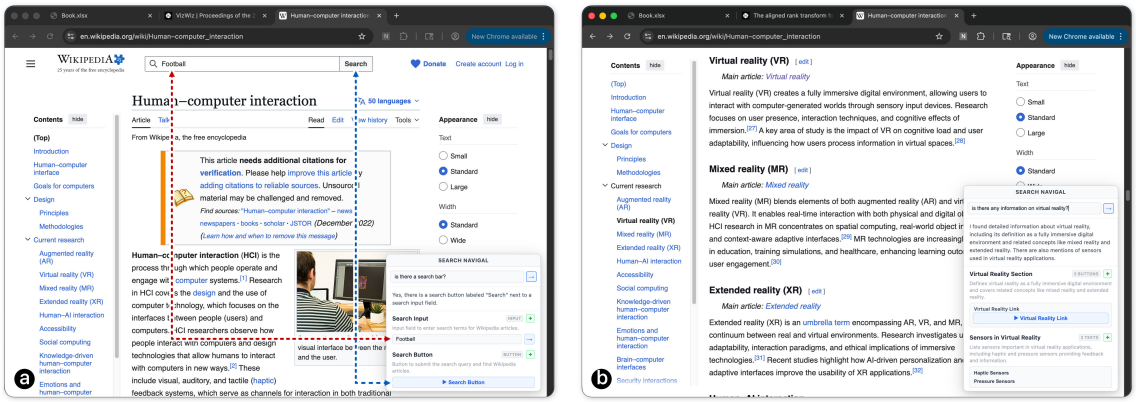



Fig. 2. Creating and using tethers illustrated on Wikipedia. (a) The user asks Navigal for a search bar, which generates a Search Input tether linked to Wikipedia's native search field. Typing "Football" into the tether automatically enters the text into the corresponding page element. (b) Tethers can also surface page content for information lookup. Pressing 'f' after navigating to a tether moves focus to its corresponding element on the page.

*3.2.1 Creating and Using Tethers.* To create tethers via natural language, the user opens the Navigal panel (Ctrl+U on macOS, Ctrl+Shift+U on Windows) and types a query describing what they need. Navigal analyzes the page and generates tethers for the matching elements, which appear as a list in the panel that the user navigates with standard screen reader controls (Figure 2a). Interacting with a tether propagates the action to the underlying page element. For controls that represent a single discrete state (buttons, checkboxes, radio options), the action propagates immediately on activation. For controls that accept continuous or textual input (text fields, sliders), a Set button lets the user confirm their entry before propagation. In Figure 2a, typing "Football" into the generated Search Input tether enters the text into Wikipedia's native search field.

Pressing 'f' after navigating to a tether moves the screen reader's focus to the corresponding source element on the page, allowing the user to inspect the surrounding context and verify that the generated control accurately reflects the page state (Figure 2b). Users can also create tethers manually by navigating to any element with their screen reader and

pressing Ctrl+K to capture it (Figure 3a), enabling quick bookmarks to points of interest that users can jump between during a browsing session. This serves a similar purpose to JAWS PlaceMarkers and VoiceOver Hot Spots, but because tethers are semantically bound rather than positionally anchored, they remain valid across page mutations and can be accessed from other tabs.

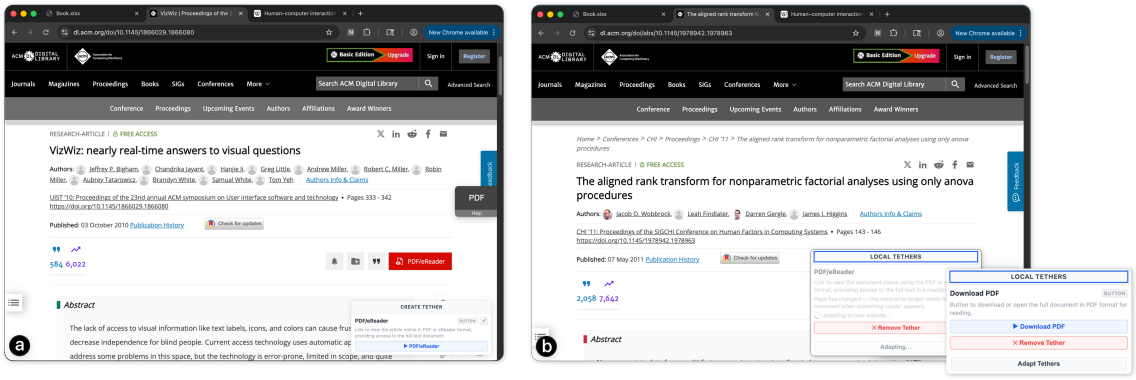



Fig. 3. Adapting tethers across pages illustrated on ACM DL. (a) The user manually captures the PDF/eReader button on an ACM Digital Library paper page as a tether. (b) Upon navigating to a different paper, Navigal detects that the original element is no longer present and adapts the tether to the new page's corresponding Download PDF button.

*3.2.2 Semantic Persistence.* Saved tethers maintain their bindings across page mutations through the semantic fingerprinting scheme described in Section 3.3.1. When a page re-renders and the underlying DOM structure changes, each tether's `navigal-id` fingerprint is matched against the new DOM to re-establish the binding. When mutations are too severe for fingerprint matching, the system activates a graduated adaptation process that escalates from deterministic metadata matching to LLM-based intent extraction and DOM search (detailed in Section 3.3).

This persistence extends beyond the current page. In Figure 3, a user manually captures the PDF/eReader button on an ACM Digital Library paper page as a tether. When they navigate to a different paper, Navigal detects that the original element is no longer present and adapts the tether to the new page's Download PDF button. The tether's function remains stable from the user's perspective while the underlying binding adapts to the changed DOM. This adaptation also works across sites altogether. A user who saves a "sort by rating" tether on one restaurant review site and later visits a different review platform can request that Navigal adapt the tether to the new context. The system extracts the tether's abstract intent (sorting results by rating) and searches the new page for a semantically equivalent control, generating an updated binding even though the two sites share no comparable/analogous structure.

*3.2.3 Container Simplification.* Web pages often present information in dense, complex regions. A single product card on an e-commerce site might contain a title, brand, image description, price, sale price, rating, review count, availability, shipping details, and action buttons, spanning over a dozen focusable elements that a screen reader user must traverse sequentially. Navigal can filter results by user-specified criteria and collapse each matching item into a single tether with a concise, goal-relevant description and an actionable control. In Figure 4a, the query "any lamp cheaper than $10?" returns only matching products, each condensed to a label, price, and an Add to Cart button. The surfaced details also adapt to reflect the user's intent. In Figure 4b, querying by rating produces tethers whose descriptions foreground star ratings alongside price, even though the price-based query did not include ratings. This reduces per-item

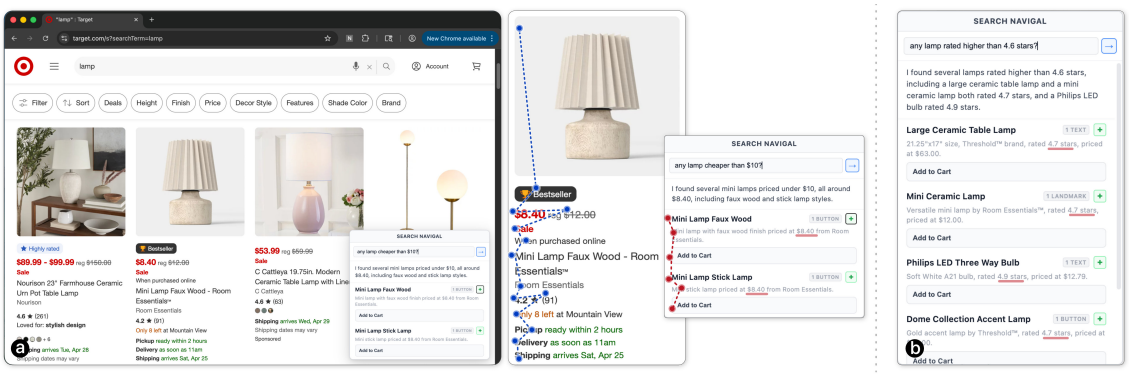



Fig. 4. Content filtering and container simplification illustrated on Target.com. (a) The user queries "any lamp cheaper than $10?" and Navigal generates tethers for matching products, each condensing the product card into a label and a brief description with price and an Add to Cart button. (b) A different query, "any lamp rated higher than 4.6 stars?", produces tethers whose descriptions foreground rating information alongside price, adapting the surfaced details to reflect the user's stated intent.

navigation from over a dozen tab stops to two or three, while ensuring the information most relevant to the user's goal is immediately available.

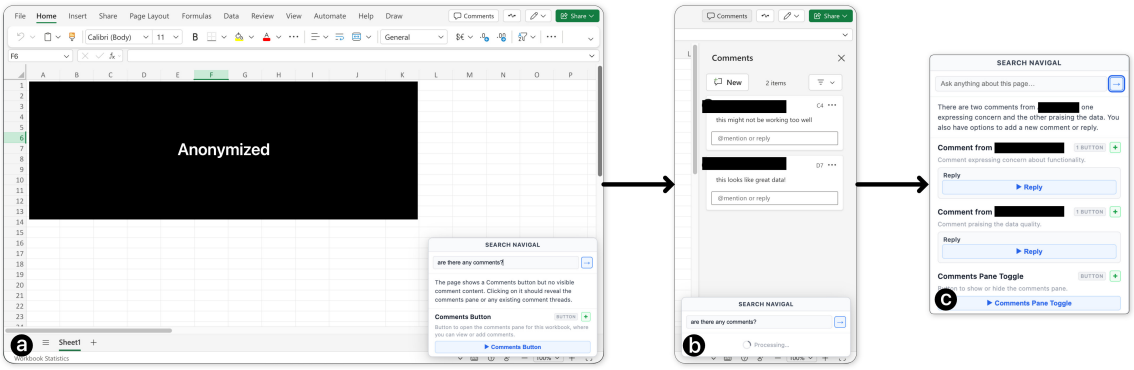



Fig. 5. Action chaining on illustrated Excel Online. (a) The user asks "are there any comments?" but the comments pane is closed. Navigal identifies the Comments button as a prerequisite and generates a tether for it. (b) Upon activating the tether, Navigal executes the prerequisite action and reprocesses the query. (c) With the comments pane now open, Navigal generates tethers for each comment with Reply buttons and a toggle to close the pane.

*3.2.4 Action Chaining.* Some page elements are not immediately accessible. Viewing comments on a shared document might require clicking a Comments button to open a pane, waiting for it to load, and then navigating through the revealed content. A screen reader user navigating conventionally would need to locate the button, activate it, wait for the pane to appear, and then find the comments in the page's linear flow. With Navigal, the user simply queries "are there any comments?" When the planner identifies that the target content requires prerequisite actions, it chains them automatically, activating the necessary control and then reprocessing the query against the updated page state. In Figure 5, the comments pane on an Excel Online spreadsheet is initially closed. Navigal identifies the Comments button as a prerequisite (a), executes it and reprocesses the query (b), and then generates tethers for each comment with reply

controls and a toggle to close the pane (c). The user receives the result without performing or even being aware of the intermediate steps.

*3.2.5 Preset Controls.* Native web controls are sometimes poorly suited to non-visual interaction. A color picker implemented as a two-dimensional canvas, a custom slider built from nested divs without ARIA attributes, or a font size selector implemented as a continuous range all present barriers for screen reader users. Navigal can generate *preset controls* that replace the native interaction modality with a more accessible alternative tailored to the user's needs. A user who queries "give me font size options for 12, 14, and 16" on a document editing page receives three button tethers, each bound to the same underlying font size control but preset to a specific value. Activating any button writes the corresponding value, replacing an inaccessible continuous control with discrete, labeled actions.

### 3.3 System Design

Navigal operates as a web browser extension that analyzes the current web page and generates tethers bound to specific page elements that are relevant to the user's query. Navigal's system workflow (Figure 6) can be broken into three primary components: (a) an ID Injector that assigns stable semantic identifiers to every meaningful element on the page, (b) a Navigal Engine that plans and executes the analysis needed to find elements relevant to the user's query, and (c) a Tether Builder that turns the engine's findings into interactive tethers in the Navigal panel. We now describe each of these components and conclude with the implementation details of our system.

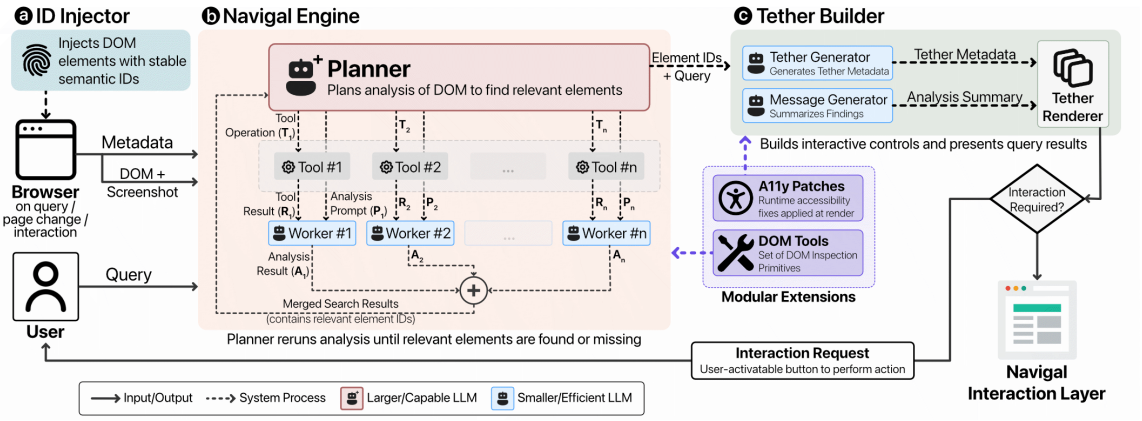



Fig. 6. System workflow of Navigal generating an accessible interaction layer from a user's query. The system comprises three primary components: (a) the **ID Injector**, which stamps DOM elements with stable semantic IDs; (b) the **Navigal Engine**, which plans and executes a search loop over the page to identify elements relevant to the user's query; and (c) the **Tether Builder**, which materializes the engine's output into interactive controls delivered to the user as the Navigal Interaction Layer.

*3.3.1 Semantic ID Injector.* To reliably bind tethers to elements of interest in the DOM, we need a way to refer to each element that holds up across page mutations. Not all elements carry a native `id` attribute, and even when they do, developers reuse or regenerate IDs in ways that make them unreliable anchors. We address this at the very start of the pipeline (Figure 6a): before any query reaches the engine, a content script walks the live DOM and stamps each semantically relevant element with a `navigal-id` attribute.

We consider an element semantically relevant if it is interactive (e.g., `<button>`, `<input>`, `<select>`), carries structural meaning (e.g., `<nav>`, `<section>`, `<main>`), has an ARIA role or label, or holds signals such as `name`, `tabindex`, or an event handler. When a qualifying element already has a native `id` attribute set by the page author, we incorporate that identifier into our fingerprint as a scoping anchor. Otherwise, we construct a fingerprint from the element's nearest landmark scope, its tag and role, its input type, its accessible name, and its link destination, and hash the result to produce a short stable token. If two elements on the page produce the same fingerprint, we disambiguate with a monotonic counter suffix assigned in document order. We also observe the DOM for changes in real time. A `MutationObserver` watches for added and removed nodes as well as attribute mutations on visibility-related properties (`style`, `class`, `aria-hidden`, `aria-expanded`), and stamps newly appearing elements as they arrive. When a previously seen element reappears after a framework re-render, we match its fingerprint against a map of known assignments and reclaim the original `navigal-id`, so any tether that referenced that element before the re-render continues to resolve correctly.

Finally, a causal tracker records the last stamped element the user interacted with and tags any DOM nodes that appear within 600 ms afterward with a `navigal-cause` attribute pointing back to that element. This makes interaction-consequence relationships explicit in the DOM, which is important for the engine to understand elements that only exist because of a prior action, such as dropdown options that appear after opening a menu or modal content that surfaces after clicking a trigger.

*3.3.2 Navigal Engine.* The Navigal Engine (Figure 6b) drives the system's core element retrieval. It takes the user's query, a screenshot of the page, additional metadata (tab title, URL, existing tether state), and a set of DOM inspection tools, and returns the elements in the DOM that may be relevant to what the user asked for.

We use a two-tier *planner-worker* architecture to handle arbitrarily large DOMs efficiently without exceeding the LLM's context window. The *planner* runs on a larger, more capable model to leverage stronger reasoning and broader world knowledge, while all *workers* run on smaller, faster models that can process directed tasks quickly. The planner receives the user's query, the page metadata, the screenshot, and descriptions of the available DOM inspection tools. From this information, it formulates a plan for each cycle of analysis: which tools to invoke against the DOM, what arguments each tool should operate over, and what analysis the worker should perform on the tool's output. These analysis instructions take the form of targeted prompts written by the planner for each tool call. Once the planner settles on a set of tool calls, the tools execute against the DOM in parallel, and each tool's result is paired with its corresponding analysis prompt and handed to an individual worker.

To illustrate, if the user is looking for a search bar on a webpage, the planner might invoke `list_interactive` to enumerate all interactive elements on the page and ask its worker to identify anything related to search functionality. In parallel, it might also invoke `search_regex` with the pattern "search" and ask a second worker to check whether any text matches correspond to a search input. Each worker receives a focused slice of the DOM (the tool's output, which is substantially smaller than the full page) along with the planner's analysis prompt, runs the requested analysis, and reports back the `navigal-id` and relevant information of any elements that match the criteria. The complete set of DOM inspection tools is documented in Appendix D.

All worker results from a given cycle are merged and returned to the planner. If the planner is satisfied that it has identified the relevant elements, or is confident that no matching elements exist on the page, it terminates and passes the collected element IDs to the Tether Builder along with the user's original query and the full trace of plans, tool invocations, and analysis results. If the planner determines that the target elements may become available through a user action (for instance, content that would appear after opening a dropdown or triggering a modal), it signals this

alongside its results. If the planner decides it has not yet gathered enough evidence, it formulates additional tool calls and the cycle repeats.

Finally, the engine also handles cases where the DOM changes underneath an existing tether. When a user navigates to a new page or the current page restructures, a tether's stored `navigal-id` may no longer resolve to any element in the DOM. We recover through a graduated process that avoids invoking the full planner unless necessary. First, we search the current DOM for elements whose metadata (accessible name, role, tag, input type) closely matches the lost tether's stored attributes, and if we find candidates with high similarity we send them to a worker to confirm the relink. If no close match surfaces, we replay the same tool calls and analysis prompts that originally produced the tether, running them against the new DOM. Because the plans and analysis instructions were saved alongside the tether's metadata, this step reuses the planner's earlier reasoning without requiring a fresh invocation, making it substantially cheaper. Only if replaying the original plan also fails do we escalate to a full planner cycle, providing it with the new DOM and the dead tether's metadata so it can search from scratch. At whichever stage a match is found, we update the tether's binding to point to the newly resolved element.

*3.3.3 Tether Builder.* Once the Navigal Engine has identified the relevant elements, the Tether Builder (Figure 6c) takes the engine's output and constructs the Navigal panel along with its tethers. The output first passes to a tether generator worker, which determines how to group elements, what metadata to produce for each tether (interaction type, label, and description), and which elements can be omitted as redundant or irrelevant to the query. The output is also sent to a message generator worker, which summarizes the engine's findings into a short natural language response. The tether generator's metadata and the message generator's summary are then passed to the renderer to render the interface [47]. The summary is displayed at the top of the panel as a direct response to the user's query, followed by the tethers themselves, each rendered from its metadata. For every tether, the renderer resolves the corresponding element's current state in the live DOM (its value, available options, range constraints) and assembles a native HTML form control that reflects that state. The renderer also creates a bi-directional bind between the tether and the corresponding element's state, so updating one reflects the value change on the other.

In some cases, the engine does not find the target elements on the page but suspects they are hidden behind an interaction. For example, a user looking for comments from a specific person on an online document may be on a page where those comments exist but are collapsed behind a "show all comments" toggle. When the engine identifies such a prerequisite action, the Tether Builder surfaces an interaction request, a button in the panel tethered to the blocking element. If the user activates that button, the corresponding action is performed on the page (expanding the section, opening the dropdown), and the engine reruns against the updated DOM, which now contains the previously hidden content and can produce the intended tethers.

*3.3.4 Implementation.* We built Navigal as a Chrome extension (Manifest V3) backed by a local Python server (Flask). The planner runs on a larger reasoning model, and all workers (the tool-result analyzers, the tether generator, and the message generator) run on a smaller, faster model, with both models interchangeable within the pipeline. A content script injects the Navigal panel into each page as a Shadow DOM widget, and we render all generated tether controls as native HTML form elements so that the browser's own accessibility tree exposes them to any screen reader without additional annotation.

The DOM tools are implemented on the backend in Python. Before any tool execution, the client sends a stripped version of the page DOM to the server, with script and style tags, media elements, data URIs, excessive URL lengths, and other non-semantic noise removed to reduce payload size. The engine then executes its chosen tools against this

cleaned representation on the backend. While our current set of DOM tools covers the majority of analysis scenarios we have encountered, the system is designed to be modular: adding a new tool requires only its implementation and a prompt description, after which the engine can begin incorporating it into its plans. The tether generation side is similarly extensible. We currently support a defined set of UI element types (Appendix E), and for each element found on the page we map it to the closest renderable equivalent, but additional custom controls can be introduced by implementing the component and extending the generation prompt. The same pattern applies to accessibility patches, such as generating alternative text for elements that lack descriptions, which can be registered without modifying the core pipeline. The system prompts governing the planner's query processing loop, tether generation, and fingerprint adaptation are provided as supplementary material, and our system is made open source at [anonymized].

## 4 Technical Evaluation

We performed a technical evaluation to assess how effectively Navigal supports BVI users in everyday web use. We sampled diverse everyday web tasks performed by BVI users from prior literature and used Navigal to complete each task, evaluating the correctness of the generated tethers and the navigation effort required to reach task-relevant elements through Navigal. To characterize the practical overhead of using Navigal, we further measured the latency and cost of running Navigal across these tasks.

### 4.1 Method

We sampled 20 everyday web tasks from those performed by BVI users in prior studies of AI-assisted web navigation [23, 42]. The sampled tasks span 19 websites across 12 domains, including shopping, news, travel, reference, productivity, communication, social media, finance, and government services, and range from single-page information lookup to multi-page goals involving form entry and multi-step interactions. The websites had a mean Google Lighthouse accessibility score of 84.7 (SD = 10.0). For each task, the experimenter opened the starting webpage and completed the task by issuing natural language queries to Navigal and operating the website through the generated tethers. The tethers were used to either directly interact with the webpage elements, or to find relevant elements on the webpage to navigate to. At each step, we saved a static snapshot of the webpage together with Navigal's output. All tasks ran with `gpt-5.6-sol` as the planner and `gpt-5.6-terra` as the worker model. Appendix A lists the full set of tasks and their corresponding queries.

### 4.2 Measures

We measured the *correctness*, *navigation effort*, and *performance overheads* of Navigal to understand how accurately and efficiently it supports BVI users in completing everyday web tasks.

*4.2.1 Correctness.* For each task, one author categorized the correctness of every tether generated by Navigal against the user's goal and the saved snapshot of the webpage, and a second author reviewed these categorizations. A tether was *correct* when it was directly relevant to the user's goal and bound to the corresponding source element on the webpage. A tether was *partially correct* when it was relevant to the user's goal but its source element was hidden on the webpage, it failed to operate its source element, it was bound to a nearby element presenting accurate information, or it only partially addressed the goal. A tether was *incorrect* when it was irrelevant to the user's goal or bound to an unrelated source element. The authors also inspected each snapshot to identify task-relevant elements that Navigal missed.

*4.2.2 Navigation Effort.* We measured navigation effort as the number of screen reader navigation steps required to reach the target elements of each task, as the number of navigation actions between interface elements is a major contributor to the perceived accessibility of an interface for screen reader users [31]. We compared Navigal against an upper and a theoretical lower bound for navigating the original webpage. The upper bound counts the steps required to reach each target element by iterating through the focusable elements of the page. BVI users commonly combine multiple navigation strategies to reach content faster [15], so the lower bound counts the steps along the shortest path to each target element using landmark, heading, form control, and rotor commands, following prior work that models reaching an element as a shortest path over the page structure [51]. For Navigal, we counted the steps required to reach the tether corresponding to each target element within the Navigal panel, excluding the keystrokes for entering queries. Target elements that lacked a native navigation path on the original webpage were accessible only through Navigal's tethers, so we excluded them from the corresponding bound.

*4.2.3 Performance Overheads.* We measured the performance of Navigal through its latency, token usage, and cost for each query. Latency was measured as the time Navigal took to generate its response, excluding the time spent entering the query. Token usage included the input and output tokens processed by both the planner and worker models, from which we calculated the cost of each query. We also recorded the DOM size of each webpage to characterize the complexity of the pages Navigal analyzed.

### 4.3 Results

Navigal supported 18 of the 20 tasks, fully completing 15 and partially completing 3. Across all tasks, Navigal generated 66 tethers, of which 59 (89.4%) were correct, 6 (9.1%) were partially correct, and 1 (1.5%) was incorrect. Most errors occurred when Navigal operated elements it had correctly identified. Four partially correct tethers were bound to the intended elements on websites with custom widget implementations, such as Reddit and Google Drive, where the widgets failed to respond to the actions Navigal propagated. The remaining errors stemmed from imprecise grounding, where Navigal bound tethers to elements adjacent to the target, hidden within a nested menu, or unrelated to the query when the requested information was absent from the webpage. Navigal also missed 4 task-relevant elements across 4 tasks, typically when a query matched many elements on the webpage and Navigal surfaced a subset of them.

Navigal reduced the navigation effort required to reach target elements compared to iterating through the page, requiring a median of 6 steps to reach the target elements of each task (range 4–14; $M = 6.44$, $SD = 2.95$). Of the 66 target elements, 27 were not keyboard focusable on the original webpage, and Navigal provided direct access to these elements through its tethers. For the remaining 39 target elements, which were reachable through all navigation methods, Navigal required a median of 6 steps (range 4–14; $M = 6.85$, $SD = 3.33$). Reaching the same elements by iterating through the focusable elements of the page (upper bound) required a median of 39 steps (range 3–450; $M = 50.49$, $SD = 71.68$), while the shortest path using structural navigation commands (lower bound) required a median of 4 steps (range 1–33; $M = 5.28$, $SD = 5.60$). The effort required by sequential navigation varied widely with page structure and the location of the target element, reaching up to 450 steps, whereas Navigal remained within 4 to 14 steps across all webpages. Navigal therefore approached the lower bound, which assumes complete knowledge of the page layout and the optimal combination of navigation commands, while Navigal only required users to describe their goal.

Navigal completed each task in a median of 34.37 seconds (range 9.62–81.17; $M = 35.04$, $SD = 19.84$) at a median cost of \$0.10 (range \$0.03–\$0.19; $M =$ \$0.10, $SD =$ \$0.05), with all 20 tasks costing \$1.99 in total. Each task consumed a median of 62,068 input tokens (range 22,952–158,415) and 1,169 output tokens (range 219–5,263). Latency and cost

grew with the number of webpages a task spanned, since Navigal analyzed each webpage separately, amounting to a median of 19.84 seconds and $0.06 per webpage. The DOM size of the webpages ranged from 57.5 KB to 1,402.9 KB (median 275.1 KB), and latency remained independent of DOM size, with Navigal processing the largest webpage, a 1.4 MB Wikipedia article, in 10.02 seconds. Appendix B reports the complete results for each task.

## 5 Exploratory User Study

We conducted an exploratory study with screen reader users to understand how they experienced using Navigal for web navigation. Participants completed structured shopping tasks using both Navigal and their screen reader alone, followed by a demonstration of the full system concept and a feedback session. Through the study, we aimed to understand (i) how Navigal affects perceived workload and browsing experience compared to screen reader navigation alone, and (ii) how participants perceive the concept of a personalized, generated navigation interface for screen reader use. The study was approved by our institution's IRB.

### 5.1 Participants

We recruited seven blind screen reader users (2 female, 5 male; ages 24–76, median 36) through snowball sampling and word of mouth. All had no functional vision, except P1 and P7 who had light perception and P6 who retained minimal residual peripheral vision. Participants had varied levels of experience using screen readers in their daily lives, spanning JAWS, NVDA, and VoiceOver, and included several accessibility professionals. Most participants also had prior experience with AI-enabled assistive tools such as Be My AI and JAWS PictureSmart. Participants reported varied online shopping experience, ranging from frequent shoppers with established comparison strategies to those who rarely purchased physical goods online. P5 and P6 also took part in the formative sessions described in Section 3.1. Participant demographics are summarized in Table 2.

Table 2. Demographics of participants in our exploratory study. All participants were blind and had varied levels of experience using screen readers in their daily lives.

| ID | Age | Gender | Vision Level | Onset | Primary SR | SR Exp. | Occupation |
|---|---|---|---|---|---|---|---|
| P1 | 24 | F | Light perception | Birth | JAWS | 12 years | Accessibility Consultant |
| P2 | 36 | F | No Vision | Birth | NVDA | 25 years | Marketing Manager |
| P3 | 76 | M | No Vision | Birth | JAWS | 39 years | Retired |
| P4 | 32 | M | No Vision | Age 5 | JAWS | 15 years | Accessibility Tester |
| P5 | 52 | M | No Vision | 2020 | JAWS | 3 years | Assistive Tech Instructor |
| P6 | 31 | M | Residual peripheral | Gradual | VoiceOver | 6 years | CS PhD Student |
| P7 | 67 | M | Light perception | 1985 | JAWS | 1 year | Social Worker |

### 5.2 Method

We conducted one-on-one remote sessions over Zoom, each lasting approximately 90 minutes. Participants completed Amazon shopping tasks under two conditions: browsing with Navigal alongside their screen reader, and browsing with their screen reader alone, in a within-subjects design with counterbalanced condition order. Two participants completed the study across two sessions due to technical difficulties. Participants were compensated with a $40 USD gift card for their time.

*5.2.1 Tasks.* For each condition, participants were given a list of three items to find on Amazon, each with specific attribute constraints (e.g., size, color, material) and a minimum price, all under a shared budget. The constraints were designed to encourage deliberate product comparison rather than arbitrary selection. We created two task variants of comparable structure and difficulty, assigning each to a different condition to mitigate learning effects. Participants were free to approach the tasks however they chose, and tasks were considered complete when the participant was satisfied with their selections. Full task specifications are provided in Appendix C.2.

*5.2.2 Procedure.* Each session began with a consent process and a short interview about participants' visual impairment, screen reader experience, and familiarity with AI-enabled assistive tools. Participants then installed the Navigal extension on their own machine with guidance from the experimenter over screen share, followed by a brief tutorial on invoking the search overlay, entering queries, browsing generated results, interacting with the tethers, and jumping to source locations on the page. Before starting the tasks, participants were invited to try Navigal on a website of their choice to familiarize themselves with the interaction. During the study, Navigal used GPT-5.4 as the planner with reasoning effort set to low and GPT-4.1-mini as the worker model at temperature 0.2.

Participants then completed both conditions in their assigned order, using whichever screen reader and Chromium-based browser they were most comfortable with on their own computer. In the Navigal condition, participants were asked to incorporate Navigal into their workflow but were free to determine how. Because our goal was to understand participants' experience with each system rather than measure unassisted performance, participants in both conditions were encouraged to ask questions and seek guidance from the experimenter. In the screen reader condition, the experimenter shared keyboard shortcuts for accessing Amazon's search bar and filter pane when participants requested help locating these controls. In the Navigal condition, participants were free to experiment with the system and ask about available interactions, and the experimenter suggested relevant capabilities when participants were unsure how to proceed. After each condition, participants completed a post-task questionnaire and reflected on their browsing experience in a short interview. Following both conditions, the experimenter demonstrated additional capabilities not covered in the tutorial, including persistent tether bindings, cross-tab access, and semantic adaptation to page changes, and conducted a semi-structured interview on participants' reactions to the broader system concept and whether they could see themselves using these features. All sessions were audio and screen recorded.

*5.2.3 Measures and Analysis.* After each condition, participants rated the six NASA-TLX subscales [26] on a 7-point scale, along with five custom items capturing navigation-specific experience (Appendix C.1). Open-ended questions after each condition asked participants to reflect on what worked well and what did not, and the final semi-structured interview gathered reactions to the demonstrated features and the system concept as a whole.

Given the small sample size ($n = 7$), we used Wilcoxon signed-rank tests for paired comparisons and report effect sizes ($r = Z/\sqrt{N}$) to capture meaningful trends across conditions. Qualitative findings were derived from thematic analysis [16] of post-task and semi-structured interview transcripts and experimenter notes. One author coded the data and discussed the codes and resulting themes with a second author. We retained only themes that recurred across multiple participants.

### 5.3 Findings

We now report findings from our exploratory study, beginning with an overview of participants' experience across both conditions and quantitative analysis of perceived workload and navigation experience, followed by qualitative themes drawn from participant interviews and follow-up responses.

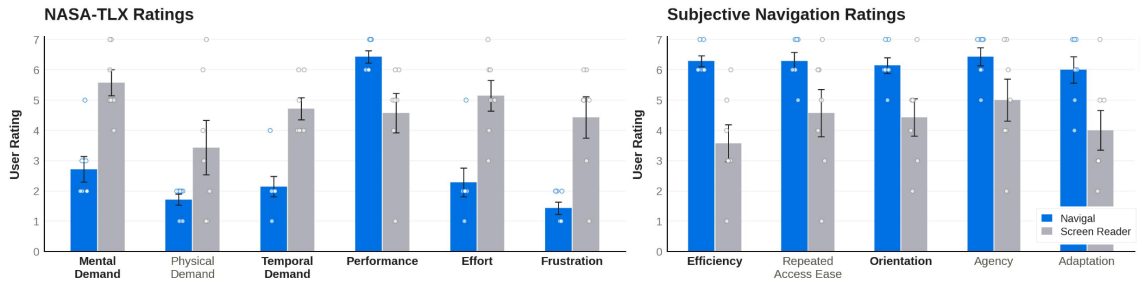



Fig. 7. Likert ratings across NASA-TLX subscales (left) and subjective navigation measures (right) for Navigal (blue) and screen reader (gray) conditions. Lower scores indicate less workload on all TLX subscales except Performance, where higher is better. Higher scores are better on all subjective navigation measures. All eleven measures favor Navigal directionally; bold labels indicate measures with statistically significant differences between conditions (Wilcoxon signed-rank, $p < .05$).

*5.3.1 Study Overview.* All seven participants successfully completed the shopping task with Navigal and strongly preferred it over screen reader navigation alone. One participant (P7) partially completed the task in the screen reader condition (finding two of three items) before the session reached its 40-minute cap. Participants described using Navigal as *easy* (P1, P2, P4, P5, P6, P7) and reported that it led to a more *accessible* (P1, P4, P7) and *personalized* (P1, P2, P4, P5) navigation experience, as Navigal generated controls to directly address their specific needs, *"The system gave me the information that I needed and nothing more or nothing less…I felt like it was really easy to navigate"* (P2). All participants also expressed interest in continuing to use the system beyond the study, *'this should be something that should be [out] there … it will make our lives much easier"* (P6).

Since tasks were open-ended and participants could approach them using whatever strategy they preferred, we observed varied approaches that evolved over the course of the session. In the screen-reader-only condition, some participants (P1, P4, P7) applied filters (e.g., shoe size, minimum price) one by one after making their initial search, while others (P2, P3, P5, P6) typed their full requirements into Amazon's search bar (e.g., black sneaker size 10 over $45) and browsed results directly. With Navigal, a similar split emerged: the former group prompted the system to surface individual filter controls, while the latter searched Amazon with their full criteria and then prompted Navigal to list matching products from the results page. In both conditions, participants converged on a hybrid approach by their later tasks, combining partial searches with selective filter application.

*5.3.2 Perceived Workload and Navigation Experience.* Figure 7 summarizes the perceived workload and navigation experience distributions across conditions. All eleven measures favored Navigal, with uniformly large effect sizes (all $r \geq .80$). Across the NASA-TLX subscales, Navigal significantly reduced Temporal Demand ($Mdn = 2.0$ vs. 4.0, $W = 0$, $p = .016$, $r = .89$), Mental Demand ($Mdn = 2.0$ vs. 5.0, $W = 1$, $p = .031$, $r = .83$), Frustration ($Mdn = 1.0$ vs. 5.0, $W = 0$, $p = .031$, $r = .89$), and Effort ($Mdn = 2.0$ vs. 5.0, $W = 1.5$, $p = .047$, $r = .80$), and significantly improved Perceived Performance ($Mdn = 6.0$ vs. 5.0, $W = 0$, $p = .031$, $r = .89$). Physical Demand showed a consistent but non-significant reduction ($Mdn = 2.0$ vs. 3.0, $p = .125$, $r = .89$).

Among the subjective navigation experience ratings, Perceived Efficiency ($Mdn = 6.0$ vs. 3.0, $W = 0$, $p = .031$, $r = .89$) and Orientation ($Mdn = 6.0$ vs. 5.0, $W = 0$, $p = .031$, $r = .89$) showed significant improvements. Repeated Access ($Mdn = 6.0$ vs. 5.0, $r = .89$), Agency ($Mdn = 7.0$ vs. 5.0, $r = .89$), and Adaptation ($Mdn = 6.0$ vs. 3.0, $r = .83$)

all trended toward Navigal, consistent with constructs that a single session is unlikely to fully surface. Mixed-model ANOVAs confirmed no significant order effects or order-by-condition interactions on any measure.

*5.3.3 Navigal Decoupled Interaction from Page Structure.* In the screen-reader-only condition, participants' ability to complete tasks depended on locating controls within Amazon's page structure. The site's filter section follows the search results in focus order rather than preceding them [60], and several participants could not find it through standard navigation. P2, a frequent Amazon shopper, had never encountered the filters: *"I literally had to type into NVDA, like, search for filters... that was the only way I could find them, otherwise I had no idea where they were"* (P2). P3 similarly noted that *"finding the right filters, and knowing what's available in the filters was challenging"* (P3). P4, after discovering the filters were placed below the results, resorted to navigating from the bottom of the page upward: *"Once I figured it out, I went on the down end of the page, and from there, I tried to figure it out, because that was the nearest place, easiest way to go there"* (P4). In several cases, the experimenter provided participants with Amazon's keyboard shopping shortcuts to access the filter section. Across participants, the interaction was fundamentally shaped by where the page author had placed controls and whether participants could discover that placement.

Participants using Navigal did not need to navigate the page's native structure to access controls. They queried the system, and relevant controls were generated and presented directly. As P2 noted, *"I didn't have to deal with going back to Amazon and dealing with the headings and stuff"* (P2). P4 described the interaction as uniform: *"You type your query and press tab, and... all the controls were generated, accessibility-wise. So, it was quite easy"* (P4). Participants also reported that Navigal's generated listings simplified how they compared products. Rather than stepping through scattered elements across each product card, participants received consolidated components containing the attributes relevant to their query. As P6 described: *"Even when [Navigal] was giving me all those lists of products, ... it felt much easier to navigate between the products. Here [screen reader], I had to, like, check every detail"* (P6). By generating controls on demand rather than requiring participants to locate them within the page, Navigal decoupled the interaction from the website's structure, providing a consistent entry point regardless of how a page was organized.

*5.3.4 Agency and Verification.* Participants expressed a clear preference for AI that surfaces controls rather than executes actions autonomously (P1, P3, P6). P6, who has experience developing AI-powered assistive technologies, articulated this most directly: *"I am a person who is really concerned about safety and security... I would not use agentic AI for this kind of tasks, especially related to the internet or shopping"* (P6). P3 similarly expressed wariness about AI autonomy, noting *"not sure I want AI to quite be that controlling"* (P3). Navigal's source verification feature, which lets users jump from a tether to its corresponding element in the original page, was valued by participants who used it. P6 noted that *"at the end of the day, it's an AI... but I can verify, I can go to the cart and verify, so that makes things a lot easier"* (P6). This was especially important in scenarios involving financial transactions: *"In these kind of cases where there is money included, I would ask someone else to check my cart first... or I would just go back to my cart and manually check"* (P6). These responses suggest that for screen reader users, who cannot visually cross-check AI output, traceability from generated controls to their source elements is a prerequisite for trust.

*5.3.5 Envisioned Applications for Personalized, Reusable Interfaces.* During the study, several participants spontaneously tried Navigal on websites where they had previously faced accessibility barriers. P5 tried Navigal on Eventbrite to locate local events, and P6 used it on ACM Digital Library to find and download a paper, noting that *"finding the PDF button has always been a tough task for me"* (P6).

Across the study, six out of the seven participants identified scenarios where they would want to create personalized interfaces that persist across sessions and transfer across sites. Several participants described habitual interactions that currently require re-navigating the same page structure on every visit: P2 named login links, restaurant menus, and add-to-cart buttons; P3 described checking current conditions on Weather.gov; P5 referenced email compose buttons and account settings. Others identified use cases where Navigal could make previously inaccessible workflows viable: P4 pointed to media controls on streaming sites and flight booking forms, while P1 described wanting quick access points on recipe websites rather than navigating through ads: *"I just wanted to get to a recipe, but it took me a long time…I can just ask it to display the recipe for me…because those websites have a ton of ads"* (P1). P6 envisioned using saved tethers to navigate within academic papers: *"I'm reading one section, I need to go back to a previous section…instead of that, I can just use that feature and go back there, read it again, and then come back to the section I was in"* (P6). P5 further identified the potential for cross-site transfer, noting that *"many sites have similar layouts…labeled differently or structured slightly differently. If Navigal could recognize those patterns and apply what I've saved before, it would make new websites much easier to use"* (P5). These responses point to a broader potential for Navigal as a platform for user-defined, persistent navigation interfaces across the web.

## 6 Discussion

Our technical evaluation showed that Navigal reliably generated accessible controls across diverse websites and brought navigation effort close to that of an expert user with complete knowledge of the page, while only requiring users to describe their goal. Our exploratory evaluation complemented these findings by examining how Navigal affects perceived workload and browsing experience relative to unassisted screen reader use, and how participants perceive the concept of a generated navigation interface. Across all eleven measures, participants reported lower workload and a better navigation experience with Navigal. In the screen-reader-only condition, task completion depended on discovering and reaching controls within the page's native structure, which was difficult even on familiar sites. With Navigal, participants instead expressed what they needed and interacted with generated controls directly, bypassing page structure altogether (**DC1**, **DC2**). Some participants voiced a preference for AI that surfaces controls rather than acts autonomously, describing source verification as important to trust, particularly in high-stakes interactions (**DC4**). This is consistent with work showing that blind users approach AI output with calibrated skepticism, such as cross-referencing across tools, testing in low-risk contexts, and seeking sighted confirmation when precision matters [2, 4, 43, 54], and with the agency concerns raised by agentic systems that execute without user involvement [42]. Beyond the structured tasks, participants identified recurring interactions from their own browsing where they would want persistent, reusable interfaces, suggesting the paradigm generalizes beyond the shopping domain we tested (**DC3**). We now discuss the broader implications of these findings.

### 6.1 Personalized Navigation for All

Navigal instantiates ability-based design [61] through intent by letting users express goals in natural language and generates matched interfaces. Our participants identified several populations beyond screen reader users whose access needs this approach could serve. P4 noted that searching for controls through Navigal would be far less demanding for motor-impaired voice-recognition users than sequential tabbing, and observed that the fixed-position panel would suit screen magnifier users who can only view a small screen portion. Thus, rather than panning the full page, they could concentrate on a single predictable region (P4). Prior work on semantically-aware magnification has shown efficiency gains from preserving local context during zoom [12], but users must still navigate within the original layout;

Navigal eliminates this by decoupling controls from page structure entirely. P5 and P6 suggested that even sighted users could benefit, noting that scrolling through websites can be tedious regardless of vision (P6). This echoes findings that both sighted and screen reader users benefit from task-relevant filtering [23], and reflects the well-documented pattern in which accessibility accommodations yield broader population benefits [1, 13, 46]. The shift Navigal embodies, i.e., users specifying *what* they need rather than *how* to reach it, has been characterized as an emerging intent-based interaction paradigm [41], and may benefit anyone whose access is constrained by page structure, whether due to disability, situational impairment [61], or attention challenges [48].

### 6.2 Towards Richer UI Primitives

Navigal's generated interfaces reconstruct controls using metadata from their source elements, producing versions that are less navigation-dense, more clearly labeled, and better suited to sequential interaction. In some cases, this resolved accessibility problems in the original page without specific engineering effort. Amazon's price range slider registers every adjustment as a discrete click that triggers a full page reload, causing screen reader users to lose focus and orientation with each incremental change. Navigal's generated version lets users set a value and confirm it with a single action, writing to the source element only on explicit submission. Participants identified further opportunities: silent page reloads with no confirmation after applying a filter (P1), the inability to type price values directly instead of manipulating a slider (P2), and filters that appear as checkboxes but function as buttons each triggering a reload, preventing batch application (P4). These observations share a common structure: confirmation feedback after state changes, input modality alternatives for controls that assume fine motor precision, and batch operations that reduce repeated page disruption. Our current implementation already demonstrates rich, custom primitives such as action chaining, where a single tether encodes a multi-step sequence like clicking a menu button and then selecting an option within it. The same mechanism could support conditional validation or composite batch operations. Defining a richer vocabulary of these patterns remains an open design question, but Navigal provides the platform on which it can be developed.

### 6.3 Extending Beyond the Browser

We implemented Navigal as a web browser extension, but the underlying paradigm is not limited to web-based environments. Desktop operating systems expose structured element hierarchies through accessibility APIs, and recent work has demonstrated that LLMs can translate natural-language commands into action sequences over these trees across arbitrary desktop applications [35, 55]. Benchmarks across full desktop environments confirm that while current models still lag behind human performance—reinforcing the value of mixed-initiative approaches where AI augments rather than replaces navigation [29, 30, 62]—they can already reliably identify and interact with interface elements. Navigal's tether model could operate over this infrastructure, with window identifiers replacing tab identifiers and accessibility tree nodes replacing DOM elements. P1 noted that she would want Navigal on her phone, where sequential traversal is even more costly. On mobile, AI-driven approaches have shown that accessibility metadata can be inferred even when native accessibility trees are incomplete [65], and a growing majority of screen reader users now prefer mobile for everyday tasks [59], suggesting the paradigm extends well beyond browser environments.

### 6.4 Limitations and Future Work

Our evaluation serves as an initial exploration of the interaction paradigm. Sessions were long enough for participants to use Navigal on structured tasks but not to introduce persistent bindings and cross-site adaptation and then observe

meaningful use of those features. Several participants initially treated Navigal as a general web search engine rather than a tool for the loaded page, suggesting the mental model shift is not immediate—consistent with evidence that prior tool experience heavily shapes expectations [15] and that blind users sometimes form plausible but inaccurate models of AI capabilities [2]. A longer-term deployment would better capture how users build fluency, manage tethers over time, and whether the paradigm holds across diverse interfaces.

Most errors in our technical evaluation arose when Navigal propagated interactions to websites built with custom widget implementations which ignored the actions Navigal issued to correctly identified elements. Supporting these custom implementations remains an important direction for interaction propagation. Navigal also occasionally surfaced a subset of task-relevant elements when a query matched many elements on a webpage, and every tether remains grounded to its source element so that users can verify the result (DC4). Much of Navigal's performance depends on its underlying off-the-shelf language models. Prior work has demonstrated strong performance of language models in semantically locating UI elements [23, 35], and Navigal's accuracy, latency, and cost stand to improve as these models become more capable, faster, and cheaper. Lastly, our technical evaluation did not directly evaluate the adaptation of saved tethers across websites. However, adaptation builds on the same generation process evaluated in this work, with Navigal searching for semantically equivalent elements guided by the context of the existing tethers.

## 7 Conclusion

We presented Navigal, an intelligent screen reader extension that enables users to generate personalized, goal-specific interfaces over arbitrary websites through natural language. Rather than requiring users to traverse page structure to locate and operate controls, Navigal lets them describe what they need and produces accessible tethers that bind to underlying DOM elements, adapt across contexts, and persist across sessions. A technical evaluation across 20 everyday web tasks showed that Navigal generated correct tethers for most target elements at a median cost of $0.10 per task. An exploratory evaluation with seven blind participants further showed uniformly lower workload and better navigation experience compared to unassisted screen reader use, with participants identifying diverse applications for the paradigm across their own browsing routines. We release Navigal as open source at [anonymous link] to support further research into personalized, persistent, and user-defined navigation for screen reader users.

## References

[1] Ali Abdolrahmani, Kevin M. Storer, Antony Rishin Mukkath Roy, Ravi Kuber, and Stacy M. Branham. 2020. Blind Leading the Sighted: Drawing Design Insights from Blind Users towards More Productivity-oriented Voice Interfaces. *ACM Trans. Access. Comput.* 12, 4, Article 18 (Jan. 2020), 35 pages. doi:10.1145/3368426

[2] Rudaiba Adnin and Maitraye Das. 2024. "I look at it as the king of knowledge": How Blind People Use and Understand Generative AI Tools. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility* (St. John's, NL, Canada) *(ASSETS '24)*. Association for Computing Machinery, New York, NY, USA, Article 64, 14 pages. doi:10.1145/3663548.3675631

[3] Faisal Ahmed, Yevgen Borodin, Andrii Soviak, Muhammad Islam, I.V. Ramakrishnan, and Terri Hedgpeth. 2012. Accessible skimming: faster screen reading of web pages. In *Proceedings of the 25th Annual ACM Symposium on User Interface Software and Technology* (Cambridge, Massachusetts, USA) *(UIST '12)*. Association for Computing Machinery, New York, NY, USA, 367–378. doi:10.1145/2380116.2380164

[4] Rahaf Alharbi, Pa Lor, Jaylin Herskovitz, Sarita Schoenebeck, and Robin N. Brewer. 2024. Misfitting With AI: How Blind People Verify and Contest AI Errors. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility* (St. John's, NL, Canada) *(ASSETS '24)*. Association for Computing Machinery, New York, NY, USA, Article 61, 17 pages. doi:10.1145/3663548.3675659

[5] Chieko Asakawa and Hironobu Takagi. 2000. Annotation-based transcoding for nonvisual web access. In *Proceedings of the Fourth International ACM Conference on Assistive Technologies* (Arlington, Virginia, USA) *(Assets '00)*. Association for Computing Machinery, New York, NY, USA, 172–179. doi:10.1145/354324.354588

[6] Vikas Ashok, Syed Masum Billah, Yevgen Borodin, and IV Ramakrishnan. 2019. Auto-Suggesting Browsing Actions for Personalized Web Screen Reading. In *Proceedings of the 27th ACM Conference on User Modeling, Adaptation and Personalization* (Larnaca, Cyprus) *(UMAP '19)*. Association for

Computing Machinery, New York, NY, USA, 252–260. doi:10.1145/3320435.3320460

[7] Vikas Ashok, Yury Puzis, Yevgen Borodin, and I.V. Ramakrishnan. 2017. Web Screen Reading Automation Assistance Using Semantic Abstraction. In *Proceedings of the 22nd International Conference on Intelligent User Interfaces* (Limassol, Cyprus) *(IUI '17)*. Association for Computing Machinery, New York, NY, USA, 407–418. doi:10.1145/3025171.3025229

[8] Natã M. Barbosa, Jordan Hayes, Smirity Kaushik, and Yang Wang. 2022. "Every Website Is a Puzzle!": Facilitating Access to Common Website Features for People with Visual Impairments. *ACM Trans. Access. Comput.* 15, 3, Article 19 (July 2022), 35 pages. doi:10.1145/3519032

[9] Jeffrey P. Bigham and Richard E. Ladner. 2007. Accessmonkey: a collaborative scripting framework for web users and developers. In *Proceedings of the 2007 International Cross-Disciplinary Conference on Web Accessibility (W4A)* (Banff, Canada) *(W4A '07)*. Association for Computing Machinery, New York, NY, USA, 25–34. doi:10.1145/1243441.1243452

[10] Jeffrey P. Bigham, Tessa Lau, and Jeffrey Nichols. 2009. Trailblazer: enabling blind users to blaze trails through the web. In *Proceedings of the 14th International Conference on Intelligent User Interfaces* (Sanibel Island, Florida, USA) *(IUI '09)*. Association for Computing Machinery, New York, NY, USA, 177–186. doi:10.1145/1502650.1502677

[11] Jeffrey P. Bigham, Irene Lin, and Saiph Savage. 2017. The Effects of "Not Knowing What You Don't Know" on Web Accessibility for Blind Web Users. In *Proceedings of the 19th International ACM SIGACCESS Conference on Computers and Accessibility* (Baltimore, Maryland, USA) *(ASSETS '17)*. Association for Computing Machinery, New York, NY, USA, 101–109. doi:10.1145/3132525.3132533

[12] Syed Masum Billah, Vikas Ashok, Donald E. Porter, and I.V. Ramakrishnan. 2018. SteeringWheel: A Locality-Preserving Magnification Interface for Low Vision Web Browsing. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) *(CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3173574.3173594

[13] Angela Glover Blackwell. 2017. The Curb-Cut Effect. *Stanford Social Innovation Review* 15, 1 (2017), 28–33. doi:10.48558/yvms-cc96

[14] Michael Bolin, Matthew Webber, Philip Rha, Tom Wilson, and Robert C. Miller. 2005. Automation and customization of rendered web pages. In *Proceedings of the 18th Annual ACM Symposium on User Interface Software and Technology* (Seattle, WA, USA) *(UIST '05)*. Association for Computing Machinery, New York, NY, USA, 163–172. doi:10.1145/1095034.1095062

[15] Yevgen Borodin, Jeffrey P. Bigham, Glenn Dausch, and I. V. Ramakrishnan. 2010. More than meets the eye: a survey of screen-reader browsing strategies. In *Proceedings of the 2010 International Cross Disciplinary Conference on Web Accessibility (W4A)* (Raleigh, North Carolina) *(W4A '10)*. Association for Computing Machinery, New York, NY, USA, Article 13, 10 pages. doi:10.1145/1805986.1806005

[16] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa

[17] Yining Cao, Peiling Jiang, and Haijun Xia. 2025. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 686, 20 pages. doi:10.1145/3706598.3713285

[18] Charles L. Chen and T. V. Raman. 2008. AxsJAX: a talking translation bot using google IM: bringing web-2.0 applications to life. In *Proceedings of the 2008 International Cross-Disciplinary Conference on Web Accessibility (W4A)* (Beijing, China) *(W4A '08)*. Association for Computing Machinery, New York, NY, USA, 54–56. doi:10.1145/1368044.1368056

[19] Javedul Ferdous, Hae-Na Lee, Sampath Jayarathna, and Vikas Ashok. 2022. InSupport: Proxy Interface for Enabling Efficient Non-Visual Interaction with Web Data Records. In *Proceedings of the 27th International Conference on Intelligent User Interfaces* (Helsinki, Finland) *(IUI '22)*. Association for Computing Machinery, New York, NY, USA, 49–62. doi:10.1145/3490099.3511126

[20] Amin Forootan. 2025. Empowering Agentic Non-Visual Web Navigation Through Tactile Controls and AI Support. https://openresearch.ocadu.ca/id/eprint/4779/

[21] Krzysztof Gajos and Daniel S. Weld. 2004. SUPPLE: automatically generating user interfaces. In *Proceedings of the 9th International Conference on Intelligent User Interfaces* (Funchal, Madeira, Portugal) *(IUI '04)*. Association for Computing Machinery, New York, NY, USA, 93–100. doi:10.1145/964442.964461

[22] Krzysztof Z. Gajos, Jacob O. Wobbrock, and Daniel S. Weld. 2007. Automatically generating user interfaces adapted to users' motor and vision capabilities. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology* (Newport, Rhode Island, USA) *(UIST '07)*. Association for Computing Machinery, New York, NY, USA, 231–240. doi:10.1145/1294211.1294253

[23] Ananya Gubbi Mohanbabu, Yotam Sechayk, and Amy Pavel. 2025. Task Mode: Dynamic Filtering for Task-Specific Web Navigation using LLMs. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility* (Denver, CO, USA) *(ASSETS '25)*. Association for Computing Machinery, New York, NY, USA, Article 89, 18 pages. doi:10.1145/3663547.3746401

[24] Simon Harper and Sean Bechhofer. 2007. SADIe: Structural semantics for accessibility and device independence. *ACM Trans. Comput.-Hum. Interact.* 14, 2 (Aug. 2007), 10–es. doi:10.1145/1275511.1275516

[25] Simon Harper and Neha Patel. 2005. Gist summaries for visually impaired surfers. In *Proceedings of the 7th International ACM SIGACCESS Conference on Computers and Accessibility* (Baltimore, MD, USA) *(Assets '05)*. Association for Computing Machinery, New York, NY, USA, 90–97. doi:10.1145/1090785.1090804

[26] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, Amsterdam, 139–183. doi:10.1016/S0166-4115(08)62386-9

[27] Jaylin Herskovitz, Andi Xu, Rahaf Alharbi, and Anhong Guo. 2023. Hacking, Switching, Combining: Understanding and Supporting DIY Assistive Technology Design by Blind People. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 57, 17 pages. doi:10.1145/3544548.3581249

[28] Jaylin Herskovitz, Andi Xu, Rahaf Alharbi, and Anhong Guo. 2024. ProgramAlly: Creating Custom Visual Access Programs via Multi-Modal End-User Programming. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) *(UIST '24)*. Association for Computing Machinery, New York, NY, USA, Article 85, 15 pages. doi:10.1145/3654777.3676391

[29] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Pittsburgh, Pennsylvania, USA) *(CHI '99)*. Association for Computing Machinery, New York, NY, USA, 159–166. doi:10.1145/302979.303030

[30] Faria Huq, Zora Zhiruo Wang, Frank F. Xu, Tianyue Ou, Shuyan Zhou, Jeffrey P. Bigham, and Graham Neubig. 2025. CowPilot: A Framework for Autonomous and Human-Agent Collaborative Web Navigation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*, Nouha Dziri, Sean (Xiang) Ren, and Shizhe Diao (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 163–172. doi:10.18653/v1/2025.naacl-demo.17

[31] Md Touhidul Islam, Donald E Porter, and Syed Masum Billah. 2023. A Probabilistic Model and Metrics for Estimating Perceived Accessibility of Desktop Applications in Keystroke-Based Non-Visual Interactions. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 43, 20 pages. doi:10.1145/3544548.3581400

[32] J. Bern Jordan, Victoria Van Hyning, Mason A. Jones, Rachael Bradley Montgomery, Elizabeth Bottner, and Evan Tansil. 2024. Information Wayfinding of Screen Reader Users: Five Personas to Expand Conceptualizations of User Experiences. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility* (St. John's, NL, Canada) *(ASSETS '24)*. Association for Computing Machinery, New York, NY, USA, Article 47, 7 pages. doi:10.1145/3663548.3688539

[33] Shinya Kawanaka, Yevgen Borodin, Jeffrey P. Bigham, Darren Lunn, Hironobu Takagi, and Chieko Asakawa. 2008. Accessibility commons: a metadata infrastructure for web accessibility. In *Proceedings of the 10th International ACM SIGACCESS Conference on Computers and Accessibility* (Halifax, Nova Scotia, Canada) *(Assets '08)*. Association for Computing Machinery, New York, NY, USA, 153–160. doi:10.1145/1414471.1414500

[34] Clemens N. Klokmose, James R. Eagan, Siemen Baader, Wendy Mackay, and Michel Beaudouin-Lafon. 2015. Webstrates: Shareable Dynamic Media. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology* (Charlotte, NC, USA) *(UIST '15)*. Association for Computing Machinery, New York, NY, USA, 280–290. doi:10.1145/2807442.2807446

[35] Satwik Ram Kodandaram, Utku Uckun, Xiaojun Bi, IV Ramakrishnan, and Vikas Ashok. 2024. Enabling Uniform Computer Interaction Experience for Blind Users through Large Language Models. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility* (St. John's, NL, Canada) *(ASSETS '24)*. Association for Computing Machinery, New York, NY, USA, Article 73, 14 pages. doi:10.1145/3663548.3675605

[36] Jonathan Lazar, Aaron Allen, Jason Kleinman, and Chris Malarkey. 2007. What Frustrates Screen Reader Users on the Web: A Study of 100 Blind Users. *International Journal of Human–Computer Interaction* 22, 3 (2007), 247–269. arXiv:https://doi.org/10.1080/10447310709336964 doi:10.1080/10447310709336964

[37] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WEBLINX: real-world website navigation with multi-turn dialogue. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Vienna, Austria, Article 1340, 50 pages.

[38] Jalal U. Mahmud, Yevgen Borodin, and I. V. Ramakrishnan. 2007. Csurf: a context-driven non-visual web-browser. In *Proceedings of the 16th International Conference on World Wide Web* (Banff, Alberta, Canada) *(WWW '07)*. Association for Computing Machinery, New York, NY, USA, 31–40. doi:10.1145/1242572.1242578

[39] Tlamelo Makati, Garreth W. Tigwell, and Kristen Shinohara. 2024. The Promise and Pitfalls of Web Accessibility Overlays for Blind and Low Vision Users. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility* (St. John's, NL, Canada) *(ASSETS '24)*. Association for Computing Machinery, New York, NY, USA, Article 38, 12 pages. doi:10.1145/3663548.3675650

[40] Farhani Momotaz, Md Ehtesham-Ul-Haque, and Syed Masum Billah. 2023. Understanding the Usages, Lifecycle, and Opportunities of Screen Readers' Plugins. *ACM Trans. Access. Comput.* 16, 2, Article 17 (July 2023), 35 pages. doi:10.1145/3582697

[41] Jakob Nielsen. 2023. AI: First new UI paradigm in 60 years. *Nielsen Norman Group* 18, 06 (2023), 2023.

[42] Yi-Hao Peng, Dingzeyu Li, Jeffrey P Bigham, and Amy Pavel. 2025. Morae: Proactively Pausing UI Agents for User Choices. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. Association for Computing Machinery, New York, NY, USA, Article 198, 14 pages. doi:10.1145/3746059.3747797

[43] Minoli Perera, Swamy Ananthanarayan, Cagatay Goncu, and Kim Marriott. 2025. The Sky is the Limit: Understanding How Generative AI can Enhance Screen Reader Users' Experience with Productivity Applications. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 1165, 17 pages. doi:10.1145/3706598.3713634

[44] Mahika Phutane, Crescentia Jung, Niu Chen, and Shiri Azenkot. 2023. Speaking with My Screen Reader: Using Audio Fictions to Explore Conversational Access to Interfaces. In *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility* (New York, NY, USA) *(ASSETS '23)*. Association for Computing Machinery, New York, NY, USA, Article 53, 18 pages. doi:10.1145/3597638.3608404

[45] Christopher Power, André Freire, Helen Petrie, and David Swallow. 2012. Guidelines are only half of the story: accessibility problems encountered by blind users on the web. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Austin, Texas, USA) *(CHI '12)*. Association for Computing Machinery, New York, NY, USA, 433–442. doi:10.1145/2207676.2207736

[46] Alisha Pradhan, Kanika Mehta, and Leah Findlater. 2018. "Accessibility Came by Accident": Use of Voice-Controlled Intelligent Personal Assistants by People with Disabilities. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) *(CHI '18)*.

Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3173574.3174033

[47] Adil Rahman and Seongkook Heo. 2023. Frappé: An Ultra Lightweight Mobile UI Framework for Rapid API-based Prototyping and Environmental Deployment. *Proc. ACM Hum.-Comput. Interact.* 7, MHCI, Article 211 (Sept. 2023), 23 pages. doi:10.1145/3604258

[48] Katta Spiel, Eva Hornecker, Rua Mae Williams, and Judith Good. 2022. ADHD and Technology Research – Investigated by Neurodivergent Readers. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) *(CHI '22)*. Association for Computing Machinery, New York, NY, USA, Article 547, 21 pages. doi:10.1145/3491102.3517592

[49] Abigale Stangl, Nitin Verma, Kenneth R. Fleischmann, Meredith Ringel Morris, and Danna Gurari. 2021. Going Beyond One-Size-Fits-All Image Descriptions to Satisfy the Information Wants of People Who are Blind or Have Low Vision. In *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility* (Virtual Event, USA) *(ASSETS '21)*. Association for Computing Machinery, New York, NY, USA, Article 16, 15 pages. doi:10.1145/3441852.3471233

[50] Wolfgang Stuerzlinger, Olivier Chapuis, Dusty Phillips, and Nicolas Roussel. 2006. User interface façades: towards fully adaptable user interfaces. In *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology* (Montreux, Switzerland) *(UIST '06)*. Association for Computing Machinery, New York, NY, USA, 309–318. doi:10.1145/1166253.1166301

[51] Hironobu Takagi, Chieko Asakawa, Kentarou Fukuda, and Junji Maeda. 2003. Accessibility designer: visualizing usability for the blind. In *Proceedings of the 6th International ACM SIGACCESS Conference on Computers and Accessibility* (Atlanta, GA, USA) *(Assets '04)*. Association for Computing Machinery, New York, NY, USA, 177–184. doi:10.1145/1028630.1028662

[52] Hironobu Takagi, Shinya Kawanaka, Masatomo Kobayashi, Takashi Itoh, and Chieko Asakawa. 2008. Social accessibility: achieving accessibility through collaborative metadata authoring. In *Proceedings of the 10th International ACM SIGACCESS Conference on Computers and Accessibility* (Halifax, Nova Scotia, Canada) *(ASSETS '08)*. Association for Computing Machinery, New York, NY, USA, 193–200. doi:10.1145/1414471.1414507

[53] Hironobu Takagi, Shin Saito, Kentarou Fukuda, and Chieko Asakawa. 2007. Analysis of navigability of Web applications for improving blind usability. *ACM Trans. Comput.-Hum. Interact.* 14, 3 (Sept. 2007), 13–es. doi:10.1145/1279700.1279703

[54] Xinru Tang, Ali Abdolrahmani, Darren Gergle, and Anne Marie Piper. 2025. Everyday Uncertainty: How Blind People Use GenAI Tools for Information Access. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 63, 17 pages. doi:10.1145/3706598.3713433

[55] Utku Uckun, Rohan Tumkur Suresh, Md Javedul Ferdous, Xiaojun Bi, I.V. Ramakrishnan, and Vikas Ashok. 2022. Taming User-Interface Heterogeneity with Uniform Overlays for Blind Users. In *Proceedings of the 30th ACM Conference on User Modeling, Adaptation and Personalization* (Barcelona, Spain) *(UMAP '22)*. Association for Computing Machinery, New York, NY, USA, 212–222. doi:10.1145/3503252.3531317

[56] Priyan Vaithilingam, Elena L. Glassman, Jeevana Priya Inala, and Chenglong Wang. 2024. DynaVis: Dynamically Synthesized UI Widgets for Visualization Editing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 985, 17 pages. doi:10.1145/3613904.3642639

[57] Markel Vigo and Simon Harper. 2013. Coping tactics employed by visually disabled users on the web. *Int. J. Hum.-Comput. Stud.* 71, 11 (Nov. 2013), 1013–1025. doi:10.1016/j.ijhcs.2013.08.002

[58] WebAIM. 2023. The WebAIM Million—The 2025 Report on the Accessibility of the Top 1,000,000 Home Pages.

[59] WebAIM. 2024. Screen Reader User Survey #10 Results. https://webaim.org/projects/screenreadersurvey10/. Accessed: 2026-03-31.

[60] Dustin Whisman. 2024. How accessible is Amazon? https://dustinwhisman.com/writing/accessibility-top-100/amazon/ Accessed: 2026-03-31.

[61] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. 2011. Ability-Based Design: Concept, Principles and Examples. *ACM Trans. Access. Comput.* 3, 3, Article 9 (April 2011), 27 pages. doi:10.1145/1952383.1952384

[62] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems* 37 (2024), 52040–52094.

[63] Yeliz Yesilada, Robert Stevens, Simon Harper, and Carole Goble. 2007. Evaluating DANTE: Semantic transcoding for visually disabled users. *ACM Trans. Comput.-Hum. Interact.* 14, 3 (Sept. 2007), 14–es. doi:10.1145/1279700.1279704

[64] Yaman Yu, Bektur Ryskeldiev, Ayaka Tsutsui, Matthew Gillingham, and Yang Wang. 2025. From Cluttered to Clear: Improving the Web Accessibility Design for Screen Reader Users in E-commerce With Generative AI. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility* (Denver, CO, USA) *(ASSETS '25)*. Association for Computing Machinery, New York, NY, USA, Article 87, 19 pages. doi:10.1145/3663547.3746353

[65] Xiaoyi Zhang, Lilian de Greef, Amanda Swearngin, Samuel White, Kyle Murray, Lisa Yu, Qi Shan, Jeffrey Nichols, Jason Wu, Chris Fleizach, Aaron Everitt, and Jeffrey P Bigham. 2021. Screen Recognition: Creating Accessibility Metadata for Mobile Applications from Pixels. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 275, 15 pages. doi:10.1145/3411764.3445186

[66] Xiaoyi Zhang, Anne Spencer Ross, Anat Caspi, James Fogarty, and Jacob O. Wobbrock. 2017. Interaction Proxies for Runtime Repair and Enhancement of Mobile Application Accessibility. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) *(CHI '17)*. Association for Computing Machinery, New York, NY, USA, 6024–6037. doi:10.1145/3025453.3025846

[67] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Vienna, Austria, Article 2538, 37 pages.

# APPENDIX

## A Technical Evaluation Tasks

Table 3. Tasks used in the technical evaluation, with the query issued to Navigal on each website.

| Task ID | Domain | Website | Navigal Query |
|---|---|---|---|
| SHOP-01 | Shopping | H-E-B | Find me the cheapest 4-pack vanilla Greek yogurt labeled as lower sugar or zero sugar. |
| SHOP-02 | Shopping | Target | Find the cheapest sweetened sparkling water. |
| NEWS-01 | News | BBC News | Search latest news on India. |
| NEWS-02 | News | Bloomberg | Show me the latest technology news. |
| TRAVEL-01 | Travel | Booking.com | Find the cheapest hotel in Chicago city center for these dates, for 2 adults and 1 room. |
| TRAVEL-02 | Travel | OpenTable | Find an Italian restaurant in Manhattan with a table for two at 7:00 PM. |
| REF-01 | Reference | Wikipedia | Which membrane within a plant cell contains the proteins responsible for photosynthesis? |
| REF-02 | Reference | Goodreads | Show me the rating and reviews for The Great Gatsby by F. Scott Fitzgerald. |
| REF-03 | Reference | [Anonymized] CS | Find [anonymized] and show me her academic titles and whether she is currently recruiting. |
| PROD-01 | Productivity | Google Docs | Show me the most recent edits made to this document. |
| PROD-02 | Productivity | Google Docs | Add page numbers to this document, starting from the first page. |
| CAL-01 | Calendar | Google Calendar | Turn off the notification for the "Weekly Research Meeting" event. |
| COMM-01 | Communication | Gmail | Most recent email about apartment inquiry. |
| SOCIAL-01 | Social | Reddit | Show me the top post from this week in r/blind. |
| SOCIAL-02 | Social | Stack Overflow | Find the accepted answer to this question and show me the code example it provides. |
| CONTENT-01 | Online content | Dailymotion | Show me the first asparagus recipe on this page. |
| CONTENT-02 | Online content | Khan Academy | Find the lesson about photosynthesis in the Biology course. |
| CLOUD-01 | Cloud storage | Google Drive | Find my resume inside Heritage Documents. |
| FIN-01 | Finance | Zacks | Show me today's Bull of the Day and Bear of the Day. |
| GOV-01 | Government | Regulations.gov | Find the public comment deadline for this proposed rule and how to submit a comment. |

## B Technical Evaluation Results

Table 4. Technical evaluation results for each task. C/P/I denotes the number of correct, partially correct, and incorrect tethers. Miss. denotes missed task-relevant elements, and Non-foc. denotes target elements that were not keyboard focusable. Navigation steps report medians across target elements reachable through all navigation methods (– indicates no such elements). Time, tokens (in thousands, input/output), and cost are summed across all webpages of a task.

| | Correctness | | | Navigation Steps | | | | Performance | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Task | Outcome | C/P/I | Miss. | Non-foc. | Upper | Lower | Navigal | Pages | DOM (KB) | Time (s) | Tokens | Cost ($) |
| SHOP-01 | Full | 3/0/0 | 0 | 1 | 30.5 | 6.5 | 5 | 2 | 614 | 40.4 | 71.7/2.2 | 0.110 |
| SHOP-02 | Full | 7/0/0 | 0 | 0 | 66 | 8 | 8 | 2 | 509 | 53.4 | 76.8/2.8 | 0.118 |
| NEWS-01 | Full | 7/0/0 | 1 | 5 | 22 | 1 | 9 | 2 | 222 | 36.1 | 76.9/1.6 | 0.131 |
| NEWS-02 | Full | 4/0/0 | 4 | 3 | 450 | 33 | 10 | 1 | 382 | 20.0 | 32.5/1.2 | 0.055 |
| TRAVEL-01 | Full | 6/0/0 | 0 | 2 | 13.5 | 2.5 | 9.5 | 2 | 245 | 60.3 | 94.4/3.9 | 0.147 |
| TRAVEL-02 | Full | 9/1/0 | 0 | 0 | 41 | 3 | 7 | 3 | 176 | 81.2 | 158.4/5.3 | 0.191 |
| REF-01 | Full | 1/0/0 | 0 | 1 | – | – | – | 1 | 1403 | 10.0 | 23.0/0.2 | 0.032 |
| REF-02 | Full | 2/0/0 | 0 | 0 | 18 | 2 | 4 | 2 | 141 | 63.0 | 87.8/4.3 | 0.127 |
| REF-03 | Full | 2/1/0 | 0 | 1 | 43.5 | 9 | 4 | 3 | 168 | 60.3 | 110.9/3.1 | 0.173 |
| PROD-01 | Full | 2/0/0 | 0 | 1 | 8 | 4 | 4 | 1 | 295 | 18.1 | 37.7/0.5 | 0.064 |
| PROD-02 | Failed | 0/0/1 | 0 | 1 | – | – | – | 1 | 295 | 19.1 | 36.0/0.5 | 0.065 |
| CAL-01 | Full | 4/0/0 | 0 | 1 | 27 | 5 | 6 | 2 | 131 | 36.4 | 79.5/1.4 | 0.120 |
| COMM-01 | Partial | 1/0/0 | 1 | 1 | – | – | – | 1 | 482 | 17.2 | 28.4/0.6 | 0.049 |
| SOCIAL-01 | Failed | 0/2/0 | 0 | 2 | – | – | – | 1 | 522 | 22.4 | 51.5/0.9 | 0.081 |
| SOCIAL-02 | Partial | 1/1/0 | 1 | 2 | – | – | – | 1 | 305 | 23.5 | 45.4/1.2 | 0.084 |
| CONTENT-01 | Full | 2/0/0 | 0 | 0 | 64.5 | 3 | 4 | 2 | 85 | 32.7 | 81.7/1.0 | 0.142 |
| CONTENT-02 | Full | 2/0/0 | 0 | 1 | 4 | 1 | 4 | 2 | 58 | 37.9 | 75.3/1.0 | 0.129 |
| CLOUD-01 | Partial | 0/1/0 | 0 | 1 | – | – | – | 1 | 368 | 39.5 | 52.4/1.6 | 0.088 |
| FIN-01 | Full | 2/0/0 | 0 | 1 | 99 | 9 | 4 | 1 | 256 | 19.7 | 30.5/0.6 | 0.051 |
| GOV-01 | Full | 4/0/0 | 0 | 3 | 9 | 4 | 8 | 1 | 62 | 9.6 | 23.2/0.6 | 0.034 |

# C User Study Details

## C.1 Subjective Navigation Questionnaire

After completing each condition, participants rated the following statements on a 7-point Likert scale (1 = Strongly Disagree, 7 = Strongly Agree):

(1) **Perceived Efficiency:** I was able to complete the task efficiently.
(2) **Ease of Repeated Access:** Accessing controls I needed to use repeatedly was easy.
(3) **Orientation:** I felt confident I knew where to find what I needed at any given point.
(4) **Agency:** I felt in control of how I navigated the website.
(5) **Adaptation:** When I moved to a new product category, I was able to quickly find and use the controls I needed.

## C.2 Study Tasks

Participants completed two shopping tasks on Amazon, one per condition, with task-condition assignment counterbalanced across participants. Both tasks required participants to find products matching specific attribute constraints, record prices, and stay within a total budget. The two task variants were designed to be structurally equivalent while covering different product domains to minimize learning effects across conditions.

*Variant A: Back to School Clothing Shopping.* Total budget: $220. Participants purchased the following items and recorded their prices:

(1) Sneakers: Size Men's 10, Color Black, Price ≥ $60
(2) Winter Jacket: Size Medium, Color Blue, Price ≥ $80
(3) Jeans: Size 32, Color Black, Price ≥ $40

*Variant B: Setting Up a Living Room.* Total budget: $200. Participants purchased the following items and recorded their prices:

(1) Area Rug: Material Wool, Pattern Geometric, Price ≥ $60
(2) Curtains: Material Polyester, Pattern Solid, Price ≥ $30
(3) Bedding Set: Material Cotton, Pattern Floral, Price ≥ $70

## D Planner Tool Set

Table 5 lists the nine tools available to the planner LLM during query processing. During tether adaptation, the planner uses the six DOM inspection tools, excluding screenshot analysis, on-page interaction, and record querying.

Table 5. Tools available to the planner LLM during query processing.

| Tool | Description |
|---|---|
| `digest` | Returns a hierarchical outline of the page at one of four detail levels, from landmarks and headings to detailed content, optionally emphasizing named sections. |
| `list_interactive` | Enumerates all interactive elements (or a filtered subset by type) as a flat index with accessible labels, types, and ARIA states. |
| `search_regex` | Finds elements whose text or accessible label matches a case-insensitive regex pattern, supporting keyword alternation and structured patterns such as prices and dates. |
| `query` | Runs a CSS selector against the page and returns matching elements for structural or attribute-based lookups. |
| `get_context` | Returns the DOM subtree surrounding a specific element, walking up a configurable number of ancestor levels and optionally including all HTML attributes. |
| `find_structures` | Retrieves repeating card or list-like DOM regions (product grids, search results, feeds) whose records match a keyword or pattern. |
| `query_records` | Extracts a typed table from a repeating region and returns records with their attributes and interactive controls. Supports sorting, numeric filtering, and limiting results. |
| `analyze_screenshot` | Analyzes the current page screenshot to assess layout, visible labels, and UI state, orienting subsequent DOM tool calls or answering visually dependent questions. |
| `request_interaction` | Proposes activating an element to reveal content (e.g., opening a dropdown or expanding a section). On user approval, performs the click and returns the updated DOM and screenshot. |

## E Tether Types

Table 6 lists the nine tether types Navigal currently supports. Each type maps a category of web interaction or content to native HTML controls, labeled groups, or descriptive text that screen readers can announce and navigate.

Table 6. Supported tether types and their rendered controls.

| Type | Description |
|---|---|
| Button | Triggers a discrete page action such as adding an item to a cart or opening a link. Rendered as a `<button>`. |
| Checkbox | Exposes an independently toggleable option and its checked state. Rendered as a labeled `<input type="checkbox">`. |
| Radio | Exposes mutually exclusive selections as a group. Rendered as `<input type="radio">` elements within a `radiogroup`. |
| Input | Accepts values such as price limits or search terms. Rendered as an `<input>` using the source input type, with a Set button or Enter key to apply the value. |
| Slider | Exposes numeric ranges with accessible labels describing the current value and bounds. Rendered as `<input type="range">`, with a Set button or Enter key to apply the value. |
| Select | Presents single- or multiple-selection options as a native menu. Rendered as `<select>` with `<option>` children. |
| Container | Groups a page region's summary and related controls into a single navigable unit. Rendered as a labeled `<div role="group">` containing child tethers and controls for moving focus to their source elements. |
| Non-interactive | Surfaces query-relevant content such as headings, paragraphs, images, lists, landmarks, and alerts. Presented through a descriptive label and text, with access to the source element. |
| Multi-action button | Executes an ordered sequence of clicks, value changes, or form submission on activation, optionally conditioned on an element's presence or visibility. Rendered as a `<button>`. |

Tethers can also carry an attached quick action, such as entering a search term and submitting it with one activation. This adds a shortcut button while leaving the base control independently usable.

## F System Prompts

The full system prompts for the planner's query processing loop, the tether generation pipeline, and the fingerprint adaptation pipeline are provided as supplementary material accompanying this submission.